

PageRank 算法实验报告

2311727 苏耀磊

一、实验目的

本实验旨在通过实现 PageRank 算法，深入理解图算法在大规模网络分析中的应用，并在此基础上掌握内存优化和高效计算的技术。具体目标包括：

- 理解 PageRank 算法的计算流程，包括考虑死端节点（Dead-ends）和蜘蛛陷阱（Spider-traps）对迭代收敛的影响，并能够实现基本的 PageRank 计算。
- 在处理大规模图数据时，学习如何通过 Block Matrix、Sparse Matrix 等方法优化内存使用，确保程序运行期间最大内存占用低于 80MB。
- 在保证内存优化的前提下，实现高效的迭代计算，使程序能够在 60 秒内完成 PageRank 计算，并获得收敛的节点排名。
- 输出 Top-100 节点及其 PageRank 分数，并通过调整参数（如 teleport 参数）观察结果变化，培养对算法参数敏感性的理解和分析能力。

二、数据集描述

接下来对于数据集文件 Data.txt，我们可以做一个简要的分析如下：

项目	数值
文件名	Data.txt
数据格式	FromNodeID ToNodeID
总边数	150,000
唯一节点数	9,500
节点 ID 范围	0 ~ 9,999
矩阵密度	0.1662%
图类型	有向图

可以把这个数据集理解成是一张有向图，每一组数据分别表示一条有向边的出发节点和指向节点，而从结点数和边数很明显能看出，该图为一个极度稀疏的有向图，简单计算一下我们能知道，总共 9500 节点，所以有 $9,500 \times 9,500 = 90,250,000$ 个可能连接，若使用密集矩阵存储，需要的内存大概是 $90,250,000 \times 8 \text{ 字节} \approx 722 \text{ MB}$ ，因此需要采用类似于稀疏与分块优化这样减少内存消耗的策略。

三、PageRank 算法原理

3.1 背景

PageRank 的基本思想其实就是每个节点代表一个网页，每条有向边代表一个超链接（从源网页指向目标网页）。想象一位虚拟的网络用户从某个随机网页开始浏览，其行为遵循以下规则：

- 跟随链接**（概率 d）：以阻尼系数的概率，从当前网页随机选择一条出链并点击进入目标网页。若当前网页有 L 条出链，则每条被选中的概率为 $1/L$ 。
- 随机跳转**（概率 1-d）：以概率 1-d 放弃跟随链接，直接在全部 N 个网页中均匀随机地选择一个新网页跳转，每个网页被选中的概率为 $1/N$ 。

经过无限长时间的浏览，用户停留在每个网页上的时间比例趋于一个平稳概率分布，这个分布正是 PageRank 向量，因此，PageRank 本质上是一个平稳马尔可夫链的稳态分布。

3.3 核心公式

设网络中有 N 个节点，节点 i 在第 k 次迭代后的 PageRank 值记为 $PR^{(k)}(i)$ 。根据随机冲浪者模型， $PR^{(k+1)}(i)$ 由两项构成：

项一：随机跳转贡献。 用户以概率 $1 - d$ 随机跳转，每个节点被等概率选中：

$$\text{Teleport 贡献} = \frac{1 - d}{N}$$

项二：来自其他节点的链接贡献。 对每个指向 i 的节点 $j \in B_i$ (B_i 表示 in-neighbors 集合)：

- 用户从 j 点击链接到达 i 的前提是：用户当前在 j (概率 $PR^{(k)}(j)$) 且选择跟随链接 (概率 d) 且选中了指向 i 的那条出链 (概率 $1/L(j)$)。

$$\text{链接贡献} = d \cdot \sum_{j \in B_i} \frac{PR^{(k)}(j)}{L(j)}$$

合并得到 PageRank 基本迭代公式：

$$PR^{(k+1)}(i) = \frac{1 - d}{N} + d \cdot \sum_{j \in B_i} \frac{PR^{(k)}(j)}{L(j)}$$

各符号的含义如下：

符号	含义
$PR^{(k)}(i)$	节点 i 在第 k 次迭代后的 PageRank 值
d	阻尼系数，标准值 0.85
N	网络中节点总数
B_i	指向节点 i 的所有节点集合 (in-neighbors)
$L(j)$	节点 j 的出度 (out-degree)，即从 j 发出的有向边数量

初始值设置为均匀分布：

$$PR^{(0)}(i) = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

3.4 矩阵形式与线性代数视角

3.4.1 从邻接矩阵到转移矩阵

首先定义有向图的**邻接矩阵** $\mathbf{A} \in \mathbb{R}^{N \times N}$ ：

$$A_{ij} = \begin{cases} 1, & \text{若存在有向边 } j \rightarrow i \\ 0, & \text{否则} \end{cases}$$

这里的下标设计是让 A_{ij} 表示从列节点 j **指向** 行节点 i ，其目的是使矩阵乘法与概率传播的自然方向一致。

进一步定义**列随机转移矩阵** $\mathbf{S}$ ：

$$S_{ij} = \frac{A_{ij}}{L(j)} = \begin{cases} \frac{1}{L(j)}, & \text{若 } j \rightarrow i \text{ 存在} \\ 0, & \text{否则} \end{cases}$$

对于每个出度非零的节点 j ， $\mathbf{S}$ 的第 j 列元素之和为 1，表示从 j 出发转移到各节点的概率分布。因此 $\mathbf{S}$ 是一个**马尔可夫转移矩阵**。

3.4.2 PageRank 的矩阵迭代式

令 $\mathbf{p}^{(k)} = [p_1^{(k)}, p_2^{(k)}, \dots, p_N^{(k)}]^T$ 为第 k 次迭代后的 PageRank 向量， $\mathbf{1}$ 表示全 1 列向量。则节点级公式可以紧凑地写成矩阵形式：

$$\mathbf{p}^{(k+1)} = \frac{1 - d}{N} \mathbf{1} + d \cdot \mathbf{S} \cdot \mathbf{p}^{(k)}$$

初始向量：

$$\mathbf{p}^{(0)} = \frac{1}{N} \mathbf{1}$$

3.5 收敛条件

PageRank 迭代收敛的数学条件是：

- 转移矩阵经过阻尼处理后，构成**随机且不可约的矩阵**（每个节点都可到达任意节点）。
- 根据 Perron-Frobenius 定理，矩阵存在唯一的主特征向量，对应的特征值为 1。

在满足上述条件下，迭代会收敛到唯一稳定解 PR^* ，即 PageRank 向量。

因此这里使用 L1 范数来判定收敛，PR 向量是一个概率分布，L1 距离直接衡量两个概率分布之间的**总变差距离**（Total Variation Distance）：

$$\|\mathbf{p}^{(k+1)} - \mathbf{p}^{(k)}\|_1 = \sum_{i=1}^N \left| p_i^{(k+1)} - p_i^{(k)} \right| < \tau$$

3.6 Dead-end 节点

Dead-end（死端节点） 指出度为 0 的节点。当随机冲浪者到达死端节点时，没有任何出链可点击，PageRank 质量在此悬空。从数学上看，死端节点 j 对应的转移矩阵列是全零列（ $L(j) = 0$ ），列的求和不为 1，导致 $\mathbf{S}$ 不再是随机矩阵。其后果是：每轮迭代后 PR 的总和不断衰减：

$$\sum_{i=1}^N p_i^{(k+1)} < \sum_{i=1}^N p_i^{(k)}$$

如果不加处理，经过多轮迭代后 PR 总和会趋近于零，计算结果完全失真。

解决方案也很简单，在每次迭代中，将**所有 dead-end 节点的当前 PR 值汇总**，然后按 $d \cdot (\text{总和})/N$ 的比例均匀分配到每一个节点。

设死端节点集合为 $D = \{j \mid L(j) = 0\}$ ，第 k 轮 dead-end 总权重为：

$$W_{\text{dangling}}^{(k)} = \sum_{j \in D} p_j^{(k)}$$

修正后的迭代公式为：

$$\mathbf{p}^{(k+1)} = \frac{1-d}{N} \mathbf{1} + d \cdot \left(\frac{W_{\text{dangling}}^{(k)}}{N} \mathbf{1} + \mathbf{S} \cdot \mathbf{p}^{(k)} \right)$$

也能比较直观地理解成将每个 dead-end 节点虚拟地连接到所有 N 个节点（包括自身），从而恢复转移矩阵的列随机性。

3.7 Spider-trap 节点

Spider-trap（蜘蛛陷阱） 指一组节点互相链接形成闭环，并且没有指向外部节点的出链；最简单的形式是一个节点仅有指向自身的自环。当随机冲浪者进入蜘蛛陷阱后，由于没有出链可以离开，会“被困”在陷阱内不断循环，导致陷阱内的节点不断累积 PageRank 质量，陷阱外的节点 PR 值趋近于零，最终的排名完全失真。

这里就是阻尼系数起作用的地方，即使所有出链都指向陷阱内部，每轮迭代中用户仍有 $(1-d)$ 的概率选择**不跟随链接**，而是随机跳转到任意节点。以单节点自环 Spider-trap 为例：

用户从 j 出发：以概率 d 跟随自环链接回到 j （滞留陷阱），以概率 $1-d$ 随机跳转到任意节点（成功逃离）。

由于 $1-d = 0.15 > 0$ ，逃离概率严格为正，质量不会无限累积。 **d 越小，逃离概率越大，分数分布越趋向均匀； d 越大，链接结构的主导作用越强**（这也是为什么在实验部分可以看到 teleport 0.70 的 Top-1 分数 0.000181 低于 teleport 0.95 的 0.000211）。

四、核心代码分析

4.1 优化路线

在本次实验中，我分析了三种方法进行 PageRank 的求解，内存占用从最初随理论数据量 $O(N^2)$ 激增，一直压减到仅需约 48MB 以下。

4.1.1. 朴素密集矩阵

- **实现思路**：该版本是最直观的设计，按算法公式直接申请一整块尺寸为 $N \times N$ 的连续内存来表示全量转移矩阵 $\mathbf{S}$ 。然后直接利用密集的矩阵乘法 ($s @ \text{old_pr}$) 进行迭代求解。
- **内存消耗与缺陷**：峰值内存极高（视节点数增加呈不可控的平方级增长）。对本实验由于大部分结点互不相连（极其稀疏的有向图），存储大量为 0 的边不仅无意义还会造成极大的内存浪费。

基本代码如下：

```
def build_dense_transition(n, src_idx, dst_idx, out_degree):
    """构建列随机密集转移矩阵 S，dead-end 列填 1/n。"""
    s = np.zeros((n, n), dtype=DTYPE)

    non_dangling = out_degree[src_idx] > 0
    src = src_idx[non_dangling]
    dst = dst_idx[non_dangling]

    weights = (1.0 / out_degree[src]).astype(DTYPE)
    np.add.at(s, (dst, src), weights)

    dangling_mask = out_degree == 0
    if np.any(dangling_mask):
        s[:, dangling_mask] = 1.0 / n

    return s


def pagerank_dense_power(s, teleport=0.85, tol=1e-8, max_iter=100):
    """密集矩阵幂迭代求解 PageRank。"""
    n = s.shape[0]
    pr = np.full(n, 1.0 / n, dtype=DTYPE)

    for _ in range(max_iter):
        old_pr = pr
        pr = (1.0 - teleport) / n + teleport * (s @ old_pr)

        diff = float(np.linalg.norm(pr - old_pr, 1))
        if diff < tol:
            break

    total = float(pr.sum(dtype=np.float64))
    if total > 0:
        pr /= total
    return pr
```

`build_dense_transition()` 函数首先创建一个全零矩阵 `s`，然后筛选出非死端节点（`non_dangling`），仅对这些节点进行列归一化处理。通过 `np.add.at(s, (dst, src), weights)` 将每条边的权重累加到矩阵中，实现列归一化。

接下来，函数处理死端节点。如果存在死端节点，则将对应的矩阵列填充为 $1/n$ ，即将这些节点均匀连接到所有节点，防止 PageRank 流失，保证迭代收敛性。

`pagerank_dense_power()` 使用 **幂迭代法（Power Iteration）** 对密集转移矩阵 `S` 计算 PageRank，首先初始化 PageRank 向量 `pr`，均匀分配初始概率 $1/n$ ，通过不断将 PageRank 向量乘以转移矩阵，并加上随机跳转概率，逐步逼近稳态分布。

在每次迭代中，函数首先保存旧的 PageRank 向量 `old_pr`，然后执行核心迭代公式。

`s @ old_pr` 计算基于转移概率矩阵的 PageRank 传播。

迭代结束后，通过 L1 范数 `diff` 判断是否收敛，当变化量小于阈值 `tol` 时提前终止。

4.1.2. 边列表一次扫描 + 分块稀疏矩阵

- **实现思路**：这里采用了“分块（Block Matrix）”与“稀疏矩阵（Sparse CSR）”机制。程序首先将整个文件的边读入一个全量的 Python 列表 `edges` 中。紧接着，按照源节点下标将边切分为若干个块分别写盘。随后，每次迭代只从磁盘动态读取一个 CSR 稀疏矩阵块加入计算，并在算完后立刻释放该块。
- **内存消耗与缺陷**：相比稠密矩阵，该版本内存峰值骤降至约 67 MB，打破了 $N \times N$ 存储带来的消耗。然而，它最大的问题是在程序最初，需要一次性读取全部 15 万条边作为 Python Tuple 放进 List。由于 Python 对象开销高，这些看似简单的边对象依然会引发一个较高的前期内存脉冲，

制约了程序应对更大图结构的潜力。

基本代码如下：

```
class BlockWriter:
    def __init__(self, block_dir, max_open):
        self.block_dir = block_dir
        self.max_open = max_open
        self.handles = OrderedDict()

    def _open(self, block_id):
        if block_id in self.handles:
            self.handles.move_to_end(block_id)
            return self.handles[block_id]

        if len(self.handles) >= self.max_open:
            _, old_handle = self.handles.popitem(last=False)
            old_handle.close()

        path = os.path.join(self.block_dir, f"block_{block_id}{EDGE_EXT}")
        handle = open(path, "ab")
        self.handles[block_id] = handle
        return handle

    def write_edge(self, block_id, src_idx, dst_idx):
        handle = self._open(block_id)
        handle.write(struct.pack("<ii", src_idx, dst_idx))

    def close(self):
        for handle in self.handles.values():
            handle.close()
        self.handles.clear()
```

这里设计了一个 `BlockWriter` 类，将边数据按块写入磁盘，同时控制同时打开的文件数量。

`block_dir` 指定了块文件的存储目录，`max_open` 限制了同时打开的文件句柄数量，`handles` 使用 `OrderedDict` 保存打开的文件句柄，同时记录访问顺序，方便实现 LRU 关闭策略。

`_open()` 方法用于安全打开或复用块文件：如果块文件已经在 `handles` 中，直接复用，并将其移动到末尾，标记为最近使用。如果打开文件数已达 `max_open` 限制，则关闭最久未使用的文件，并从 `handles` 中移除。打开块文件时使用二进制追加模式 `"ab"`，保证边数据被追加到文件末尾而不覆盖。最终返回文件句柄供写入使用。

整个实现类似于一个文件句柄的 LRU 缓存，在处理大规模图时非常高效。

`write_edge()` 方法负责将单条边写入对应块文件：调用 `_open(block_id)` 获取对应块文件句柄。使用 `struct.pack(...)` 将两个整数打包成二进制数据写入文件。

`close()` 方法没什么好说的，就是遍历所有打开的文件句柄并关闭，清空 `handles` 字典，确保类实例退出时不残留资源。

4.1.3 两遍扫描流式分块

- **实现思路：**这是本次实验的最后的优化版本。在上一个方法的架构上对起初读取图数据的方式做了拆除与重建，使用“两遍扫描”改进法。第一遍仅统计全图节点的出度信息但不存边，结束后立刻进行强制垃圾清理 `gc.collect()`；第二遍读取时不再构建任何全量边列表，而是对逐条读入的边直接分配进各块专属的小 `Buffer` 中，当缓存达到极小的阈值（如 5 万条）时即刻化为底层类型写入磁盘。另外迭代中也辅以了诸如复用结果数组和局部掩码的一些优化。
- **内存消耗与优点：**最终把内存严格控制并削平到了 48 MB 左右，并且迭代速度也并未下降。

详细代码分析见后。

对比：

数据构建与存储方式	内存峰值	执行时间
完整边构建 + 密集矩阵 $N \times N$	734.66 MB	1.01s
全量 List 暂存边 + 分块稀疏 CSR	64.97 MB	0.81s
两遍流式扫描 + 分块稀疏 CSR	47.43 MB	0.91s

4.2 两次扫描分块稀疏 代码分析

4.2.1 read_and_build_blocks()

这个函数作为读取数据集的函数，通过两遍扫描把原始边数据变成后续 PageRank 迭代所需的分块稀疏文件，同时尽量避免把全部边一次性装入内存。

```

def read_and_build_blocks(file_path, block_size):
    """两遍读取：先统计，再按块写边文件。"""
    nodes = set()
    out_deg_dict = {}

    with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 2:
                continue
            try:
                u, v = map(int, parts[:2])
            except ValueError:
                continue
            if u < 0 or v < 0:
                continue
            nodes.add(u)
            nodes.add(v)
            out_deg_dict[u] = out_deg_dict.get(u, 0) + 1

    if not nodes:
        raise ValueError("输入文件中没有有效边")

    node_list = sorted(nodes)
    n = len(node_list)

    node_to_idx = {node: i for i, node in enumerate(node_list)}
    out_degree = np.zeros(n, dtype=np.int32)
    for node, deg in out_deg_dict.items():
        out_degree[node_to_idx[node]] = deg

    del nodes, out_deg_dict
    gc.collect()

    block_dir = tempfile.mkdtemp(prefix="pr_blocks_")
    num_blocks = (n + block_size - 1) // block_size
    buffers = [[] for _ in range(num_blocks)]
    flush_limit = 50000

    with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 2:
                continue
            try:
                u, v = map(int, parts[:2])
            except ValueError:
                continue
            if u < 0 or v < 0:
                continue

            src_idx = node_to_idx[u]
            dst_idx = node_to_idx[v]
            bid = src_idx // block_size
            buffers[bid].append((src_idx, dst_idx))

            if len(buffers[bid]) >= flush_limit:
                arr = np.array(buffers[bid], dtype=np.int32)
                with open(os.path.join(block_dir, f"block_{bid}.bin"), "ab") as fh:
                    fh.write(arr.tobytes())
                buffers[bid].clear()

```

```

for bid, buf in enumerate(bufers):
    if buf:
        arr = np.array(buf, dtype=np.int32)
        with open(os.path.join(block_dir, f"block_{bid}.bin"), "ab") as fh:
            fh.write(arr.tobytes())

del node_to_idx, buffers
gc.collect()

return node_list, out_degree, block_dir, n

```

我们来看这部分代码的具体实现方式，这里分成两次读取文件，第一遍只做统计，不保存边本身：

- `nodes = set()` 只保存出现过的节点编号，用来建立节点全集。
- `out_deg_dict = {}` 只记录每个源节点的出度，用来后续处理 `dangling` 节点和计算列归一化权重。

第一遍结束后，代码把节点编号映射成连续下标：

- `node_list = sorted(nodes)` 用来固定节点顺序。
- `node_to_idx = {node: i ...}` 把稀疏的原始节点 ID 压缩成 $0 \sim n-1$ 的连续索引，便于用 NumPy 数组存储。
- `out_degree = np.zeros(n, dtype=np.int32)` 用整数数组保存出度。

这一步之后立刻执行 `del nodes, out_deg_dict` 和 `gc.collect()`，在第二遍扫描开始前释放第一遍临时对象，避免两个阶段的数据同时驻留在内存中。

第二遍读取时，函数不再构造完整边列表，而是按源节点所在块进行分发：

- `bid = src_idx // block_size` 将边归入对应块。
- `buffers[bid].append((src_idx, dst_idx))` 只在当前块的缓冲区中暂存少量边。
- 当缓冲区达到 `flush_limit` 时，立即用 `np.array(..., dtype=np.int32)` 转成紧凑的定长数组，再批量写入磁盘。

4.2.2 build_sparse_blocks()

接下来是建立稀疏矩阵块，进行数据预处理的过程。


```

def build_sparse_blocks(block_dir, sparse_dir, block_size, n, out_degree):
    """把每个 .bin 块转成列归一化 CSR 稀疏矩阵。"""
    os.makedirs(sparse_dir, exist_ok=True)

    for fname in sorted(os.listdir(block_dir)):
        if not fname.endswith(".bin"):
            continue

        fpath = os.path.join(block_dir, fname)
        block_id = int(fname[len("block_") : -len(".bin")])

        data = np.fromfile(fpath, dtype=np.int32)
        if data.size == 0:
            continue

        data = data.reshape(-1, 2)
        src = data[:, 0]
        dst = data[:, 1]

        block_start = block_id * block_size
        block_end = min(block_start + block_size, n)
        block_width = block_end - block_start

        cols = src - block_start
        weights = (1.0 / out_degree[src]).astype(np.float32)

        mat = sp.csr_matrix(
            (weights, (dst, cols)),
            shape=(n, block_width),
            dtype=np.float32,
        )

        out_path = os.path.join(sparse_dir, f"block_{block_id}.npz")
        sp.save_npz(out_path, mat)

        del data, src, dst, cols, weights, mat

    for fname in os.listdir(block_dir):
        if fname.endswith(".bin"):
            os.unlink(os.path.join(block_dir, fname))

    gc.collect()

```

这里按块顺序遍历 `block_dir` 目录下的 `.bin` 文件，使用 `np.fromfile` 高效地加载数据，并通过 `reshape(-1, 2)` 将边数据转换为 `(src, dst)` 格式的二维数组。随后，函数根据块编号计算该块在全局节点索引中的起始和结束位置，同时将全局节点索引映射到块内列索引，为构建块矩阵做准备。

接下来是列归一化，函数根据源节点的出度计算每条边的权重，`weights = 1.0 / out_degree[src]`，保证每个节点的出链贡献总和为 1，并将数据转换为 `float32` 类型，接着使用 `scipy.sparse.csr_matrix` 构建稀疏矩阵，其中行索引为目标节点，列索引为块内节点索引，矩阵形状为 `(n, block_width)`。

矩阵构建完成后，函数将每个块保存为 `.npz` 文件，后续 PageRank 迭代中按块加载，同时为了避免内存占用累积，函数在每块处理完成后删除中间变量，并调用垃圾回收。处理完所有块后，删除原始 `.bin` 文件。

4.2.3 pagerank()

接下来是PageRank的核心计算函数：

```
def pagerank(sparse_dir, n, block_size, out_degree, teleport=0.85, tol=1e-8, max_iter=100):
    """PageRank 迭代（含 dead-end 处理）。"""
    pr = np.full(n, 1.0 / n, dtype=np.float64)
    dangling_mask = out_degree == 0

    block_paths = sorted(
        os.path.join(sparse_dir, f)
        for f in os.listdir(sparse_dir)
        if f.endswith(".npz")
    )

    for _ in range(max_iter):
        old_pr = pr.copy()

        dangling_weight = float(old_pr[dangling_mask].sum(dtype=np.float64))
        base = (1.0 - teleport) / n + teleport * dangling_weight / n
        new_pr = np.full(n, base, dtype=np.float64)

        for bp in block_paths:
            block_id = int(os.path.basename(bp)[len("block_") : -len(".npz")])
            mat = sp.load_npz(bp)
            start = block_id * block_size
            end = start + mat.shape[1]
            new_pr += teleport * (mat @ old_pr[start:end])
            del mat

        diff = float(np.linalg.norm(new_pr - old_pr, 1))
        pr = new_pr
        if diff < tol:
            break

    return pr
```

这个函数完成了稀疏矩阵块的迭代计算，并处理了 **dead-end** 和 **spider-trap** 问题，首先在函数开始时，初始化 PageRank 向量 `pr`，将所有节点的初始分值设置为均匀分布 $1/n$ 。同时，通过 `dangling_mask = out_degree == 0` 标记出度为 0 的节点，即死端节点。

接着，需要列出稀疏矩阵块的路径 `block_paths` 并按文件名排序，保证矩阵块顺序与节点索引一致，每个矩阵块存储为 CSR 格式，后续迭代时会按块加载，降低内存峰值。

在每次迭代中，首先保存旧的 PageRank 向量 `old_pr`。然后计算死端节点的总 PageRank `dangling_weight`，并将其均匀分配到所有节点上，同时加上随机跳转概率 $(1 - \text{teleport})/n$ ，得到迭代基础值 `base`。

核心迭代步骤是按块累加矩阵对 PageRank 的贡献，遍历每个稀疏矩阵块，加载矩阵 `mat`，并使用矩阵向量乘法 `mat @ old_pr[start:end]` 计算块内节点对全局节点的贡献，然后乘以阻尼因子 `teleport` 并累加到 `new_pr`。

每轮迭代结束后，通过 L1 范数 `diff = np.linalg.norm(new_pr - old_pr, 1)` 判断 PageRank 向量是否收敛。当变化量小于设定阈值 `tol` 时，迭代提前终止，保证计算效率。迭代次数最大为 `max_iter`，防止异常情况下无限循环，最终返回收敛后的 PageRank 向量 `pr`。

五、实验结果

以下为 `teleport = 0.85`、`block size=5000`、`tol=1e-8` 时的 Top-100 节点部分结果：

Rank	NodeID	PageRank Score
1	75	0.000198721375231
2	8686	0.000190205495775
3	9678	0.000188337066154
4	5104	0.000187795362583
5	725	0.000186793181237
6	3257	0.000185721659839
7	468	0.000184900213538
8	7730	0.000183938408695
9	7175	0.000183838945160
10	5526	0.000182843442091
...	...	...
100	5206	0.000158829992415

六、参数敏感性分析

6.1 Teleport 参数变化的影响

Teleport 参数是 PageRank 的核心超参数。增大 d 使分数更依赖链接结构，减小 d 使分数趋近均匀分布。

6.1.1 数值结果对比 (block size=10000, tol=1e-8)

Teleport	迭代次数	Top-1 NodeID	Top-1 Score	Top-100 Sum	PR 总和
0.70	11	75	0.00018118	0.01565318	1.00000004
0.75	11	75	0.00018697	0.01603472	1.00000005
0.80	11	75	0.00019282	0.01641945	1.00000006
0.85	12	75	0.00019872	0.01680799	1.00000009
0.90	12	75	0.00020469	0.01720003	1.00000011
0.95	13	75	0.00021071	0.01759585	1.00000016

6.1.2 排名一致性分析 (与 teleport=0.85 基准对比)

Teleport	Top-100 重叠数	Spearman 相关系数	L1 差异
0.70	96 / 100	0.9956	2.94×10^{-2}
0.75	96 / 100	0.9980	1.97×10^{-2}
0.80	98 / 100	0.9995	9.88×10^{-3}
0.90	99 / 100	0.9995	9.97×10^{-3}
0.95	97 / 100	0.9979	2.00×10^{-2}

6.1.3 结果分析

从实验结果可以看出，Teleport 参数对 PageRank 排名结果有一定影响，但总体排名具有较强的稳定性。随着 Teleport 参数从 0.70 增加到 0.95，PageRank 向量的收敛迭代次数略有增加，从 11 轮增至 13 轮。这说明增大阻尼因子会使得 PageRank 更多依赖于图的链接结构，从而需要更多迭代才能达到收敛。

在数值上，Top-1 节点的 PageRank 分值随着 Teleport 的增加呈现稳定上升趋势，从 0.00018118 增至 0.00021071，而 Top-100 节点分值总和也随之增加，这也成功验证了较大的 Teleport 参数增强了核心节点的权重，使得网络中高连接节点的 PageRank 更加突出，同时整体概率仍然严格归一化，PR 总和基本维持在 1。

从排名一致性分析来看，不同 Teleport 参数下 Top-100 节点与 Teleport=0.85 基准相比，重叠数量高达 96–99 个，Spearman 相关系数大于 0.995，L1 差异极小（均在 10^{-2} 量级），我们能够知道，尽管 PageRank 分值略有变化，但节点排名整体保持高度一致，核心节点的排序对 Teleport 参数相对不敏感。

6.2 Block Size 变化的影响

Block size 影响分块数量和每次迭代中单块加载的 I/O 开销，但 **完全不影响数值结果**。

6.2.1 实测数据 (teleport=0.85, tol=1e-8)

Block Size	块数量	迭代次数	Top-1 Score	运行时间	峰值内存
500	19	12	0.000198721375	1.51 s	53.81 MB
1,000	10	12	0.000198721375	1.21 s	52.07 MB
2,000	5	12	0.000198721375	1.01 s	50.18 MB
5,000	2	12	0.000198721375	0.91 s	47.43 MB
10,000	1	12	0.000198721375	0.91 s	48.38 MB
1,000,000	1	12	0.000198721375	0.85 s	48.30 MB

6.2.2 结果分析

实验结果显示，Block Size 对 PageRank 的计算性能和内存使用有明显影响，但对计算结果本身（Top-1 Score 和迭代次数）几乎没有影响。无论块大小从 500 变化到 1,000,000，Top-1 节点的 PageRank 分值保持稳定，迭代次数均为 12，说明分块策略不会改变 PageRank 的最终收敛结果。

在性能方面，随着 Block Size 的增大，块数量显著减少，迭代时间呈现下降趋势。从最小块 500 对应 19 块，耗时 1.51 秒，到最大块 1,000,000 对应 1 块，耗时仅 0.85 秒。这说明块过多会增加文件 I/O 和分块矩阵加载开销，而适当增大块大小可以减少这些开销，从而提升整体计算速度。

在内存占用方面，峰值内存随块大小增加而略有下降，从 53.81 MB 降至约 48 MB，说明块数量减少能够降低同时在内存中存在的稀疏矩阵块数量，从而降低内存压力。但当块大小达到一定程度（如 10,000 或更大）时，内存占用趋于稳定，块大小对内存使用的边际效果减小。

综合来看，当使用小块使得块数量多时，I/O 开销大，内存占用稍高，但粒度细，可灵活处理更大图，而大块数量少，计算速度快，内存占用略低，但单块矩阵较大时可能增加瞬时内存压力。

七、实验总结

本次实验围绕 PageRank 算法在大规模稀疏有向图上的高效实现，经历了从朴素密集矩阵到最终两遍扫描分块稀疏矩阵的三轮迭代优化过程。

在算法层面，我完整实现了 PageRank 的迭代求解，并针对 Dead-end 节点采用了 dangling 权重回传策略，利用 Teleport 机制解决了 Spider-trap 问题，确保了迭代过程的正确收敛。在优化层面，我设计了三种方案：首先是最直观的密集矩阵法，虽然直接但受限于 $O(N^2)$ 的内存增长；然后是引入了分块矩阵与稀疏 CSR 存储的版本，通过将矩阵切分并逐块加载释放，大幅度减少了内存消耗；最后通过两遍扫描彻底消除了中间边列表的内存占用，结合批量缓冲写盘和显式垃圾回收，将峰值内存进一步压缩，同时保持了约 0.9 秒的迭代速度。

之后通过对照测试分析了核心参数对结果的影响，Teleport 参数从 0.70 到 0.95 的对照实验表明，增大阻尼系数会使 PageRank 分值更向核心节点集中，但节点排名的总体结构足够稳定，Spearman 相关系数均在 0.995 以上，Top-100 重叠度不低于 96%，而 Block Size 的对照实验验证了所有块大小下的计算结果精确一致，且块大小仅通过 I/O 次数和粒度影响性能，总体来说，本次实验让我收获颇丰。